CSCI 112 Assignment – C Language Fundamentals

Points: 10

Part A — Lab Instructions

Submission: Canvas Quiz — Lab 2

Objective

Practice the following topics by writing C code from plain-English instructions:

- `printf` formatting
- Integer arithmetic and modulo
- Debugger — breakpoints and Locals panel
- Casting — implicit and explicit
- Scope and variable visibility
- Logical and relational operators
- Unsigned types and wraparound
- Prefix and postfix operators

Setup

1. Create a new C project in your IDE
2. Replace the contents of `main.c` with the provided `skeleton.c`
3. Compile — **zero errors, zero warnings** before you start
4. Open **Canvas Quiz — Lab 3** in a separate window
5. Work through sections **1 through 7** in order

How this lab works

Section 1 is the only section with `printf` already written. You will not write any `printf` calls in sections 2 through 7. All variable values in those sections are observed exclusively through the **debugger Locals panel**.

For every section from 2 to 7: read the plain-English steps, translate each one into a valid line of C code, then use the debugger to observe the result and answer the quiz questions.

Debugger — Quick Reference

Action	How
Set a breakpoint	Click to the left of a line number — a red dot appears
Start debugging	F5
Execute one line	F10 — Step Over
Read variable values	**Locals panel** at the bottom of the screen

> A variable appears in the Locals panel only **after** the line that declares it has been executed. Press F10 to land **on** a line, then press F10 once more to execute it — then read the Locals panel.

Section Overview

Section	Your task
1 — printf	Run the program. Observe the output. Modify one format specifier and observe the change. Answer Q1–Q2.
2 — Arithmetic	Translate 8 steps into C. Use the debugger to observe results. Answer Q3–Q5.
3 — Debugger / sizeof	Translate 10 steps into C. Read every variable from the Locals panel. Answer Q6–Q7.
4 — Casting	Translate 11 steps into C. Observe implicit and explicit conversions in Locals. Answer Q8–Q11.
5 — Scope	Translate 7 steps into C. Watch `val` appear, change, and disappear in Locals. Answer Q12–Q13.
6 — Logical operators	Translate 9 steps into C. Record all results from Locals. Answer Q14–Q15.
7 — Unsigned / prefix / postfix	Translate 11 steps into C. Observe wraparound and the difference between `i++` and `++i`. Answer Q16–Q20.

PART B — skeleton.c

```
#include <stdio.h>
```

```c
int main()
{
    /* ================================================================
       SECTION 1  |  printf and format specifiers

       This section is already written for you.
       Run the program and observe the full output.
       Then modify the line that uses %.2f:
       change it to %.0f, run again, and observe what changes.
       Answer Q1-Q3 from what you see.
    ================================================================ */

    char  name[]  = "Grace Hopper";
    int   year    = 1906;
    float score   = 97.5f;
    int   rank    = 1;

    printf("================================\n");
    printf("  Name:   %s\n",    name);
    printf("  Year:   %d\n",    year);
    printf("  Score:  %.2f\n",  score);   /* <-- modify this line for Q1 */
    printf("  Rank:   %05d\n",  rank);
    printf("================================\n");
    printf("Col1\tCol2\tCol3\n");
    printf("%d\t%d\t%d\n", 1, 2, 3);
    printf("score with %%f   : %f\n",   score);
    printf("score with %%.1f : %.1f\n", score);
    printf("score with %%.4f : %.4f\n", score);


    /* ================================================================
       SECTION 2  |  Integer arithmetic

       Translate each step into one line of C code.
       Place a breakpoint on Step 1.
       Press F10 to step line by line.
       Read values from the Locals panel after each step.
       Answer Q4-Q6 from what you observe.

       Step 1.  Declare an integer variable named  a  with value 7.
       Step 2.  Declare an integer variable named  b  with value 2.
       Step 3.  Declare an integer variable named  r1  that stores
                the result of  a  divided by  b.
       Step 4.  Declare an integer variable named  r2  that stores
                the remainder when  a  is divided by  b.
       Step 5.  Declare a float variable named  fa  with value 7.0f.
```

```
   Step 6.  Declare a float variable named  r3  that stores
            the result of  fa  divided by  b.
   Step 7.  Declare an integer variable named  r4  that stores
            the result of 15 divided by 4.
   Step 8.  Declare an integer variable named  r5  that stores
            the remainder when 15 is divided by 4.
============================================================ */

/* --- write your Section 2 code here --- */


/* ============================================================
   SECTION 3  |  Debugger and sizeof

   Translate each step into one line of C code.
   Place a breakpoint on Step 1.
   Press F10 to step through all lines.
   Read ALL values from the Locals panel.
   Answer Q7-Q9 from what you observe.

   Step 1.  Declare an integer variable named  p  with value 10.
   Step 2.  Declare an integer variable named  q  with value 4.
   Step 3.  Declare an integer variable named  d1  that stores
            the sum of  p  and  q.
   Step 4.  Declare an integer variable named  d2  that stores
            the integer division of  p  by  q.
   Step 5.  Declare an integer variable named  d3  that stores
            the remainder of  p  divided by  q.
   Step 6.  Declare an integer variable named  sz_char  that
            stores the number of bytes occupied by type  char.
            Use the  sizeof  operator.
   Step 7.  Declare an integer variable named  sz_short  that
            stores the number of bytes occupied by type  short.
   Step 8.  Declare an integer variable named  sz_int  that
            stores the number of bytes occupied by type  int.
   Step 9.  Declare an integer variable named  sz_float  that
            stores the number of bytes occupied by type  float.
   Step 10. Declare an integer variable named  sz_double  that
            stores the number of bytes occupied by type  double.
============================================================ */

/* --- write your Section 3 code here --- */


/* ============================================================
   SECTION 4  |  Casting
```

```
   Translate each step into one line of C code.
   Place a breakpoint on Step 1.
   Press F10 to step through all lines.
   Read values from the Locals panel after each step.
   Answer Q10-Q13 from what you observe in the Locals panel.

   Step 1.  Declare an integer variable named  ci  with value 5.
   Step 2.  Declare a float variable named  cf  with value 2.7f.
   Step 3.  Declare an integer variable named  c1  that stores
            the result of  ci + cf.
   Step 4.  Declare a float variable named  c2  that stores
            the result of  ci + cf.
   Step 5.  Declare an integer variable named  c3  that stores
            cf  cast explicitly to  int.
            Write the cast operator as:  (int)cf
   Step 6.  Declare a float variable named  c4  using
            the expression:  (float)ci / 2
   Step 7.  Declare a float variable named  c5  using
            the expression:  (float)(ci / 2)

   Step 8.  Declare a char variable named  ch  with value 'A'.
   Step 9.  Declare an integer variable named  ch_int  that
            stores  ch.
   Step 10. Declare a char variable named  ch_lower  that
            stores  ch + 32.
   Step 11. Declare an integer variable named  ch_lower_int
            that stores  ch_lower.
============================================================ */

/* --- write your Section 4 code here --- */


/* ============================================================
   SECTION 5  |  Scope and variable visibility

   Translate each step into C code.
   Place a breakpoint on Step 1.
   Press F10 to step through every line including { and }.
   Watch the Locals panel: the variable  val  will appear,
   change its value, and disappear as you move through blocks.
   Answer Q14-Q15 from what you observe.

   Step 1.   Declare an integer variable named  val  with value 10.
             >>> LOCALS: note the value of val          (POINT A)
   Step 2.   Open a code block  {
```

```
   Step 3.     Declare an integer variable named  val  with value 20.
               >>> LOCALS: note the value of val        (POINT B)
   Step 4.     Open another code block  {
   Step 5.       Declare an integer variable named  val  with value 30.
                 >>> LOCALS: note the value of val      (POINT C)
   Step 6.     Close the inner block  }
               >>> LOCALS: note the value of val        (POINT D)
   Step 7.   Close the outer block  }
             >>> LOCALS: note the value of val          (POINT E)
============================================================== */

/* --- write your Section 5 code here --- */


/* ============================================================
   SECTION 6  |  Logical and relational operators

   Translate each step into one line of C code.
   Place a breakpoint on Step 1.
   Press F10 to step through all lines.
   Read ALL values from the Locals panel.
   Answer Q16-Q17 from what you observe.

   Step 1.  Declare three integer variables on one line:
            lg_a = 5,   lg_b = 0,   lg_c = -3
   Step 2.  Declare  int  res1  =  lg_a && lg_b
   Step 3.  Declare  int  res2  =  lg_a || lg_b
   Step 4.  Declare  int  res3  =  !lg_a
   Step 5.  Declare  int  res4  =  !lg_b
   Step 6.  Declare  int  res5  =  !!lg_a
   Step 7.  Declare  int  res6  =  lg_c && lg_a
   Step 8.  Declare  int  res7  =  ( lg_a > 3 )
   Step 9.  Declare  int  res8  =  ( lg_a == lg_a )
============================================================== */

/* --- write your Section 6 code here --- */


/* ============================================================
   SECTION 7  |  Unsigned types, prefix and postfix

   Translate each step into C code.
   Place a breakpoint on Step 1.
   Press F10 to step through every line.
   Read ALL variables in Locals at each step.
   Answer Q18-Q22 from what you observe.
```

```
    Part A - Unsigned wraparound:

    Step 1.   Declare  unsigned int    u_i  =  0
    Step 2.   Declare  unsigned char   u_c  =  0
    Step 3.   Declare  unsigned short  u_s  =  0
    Step 4.   Decrement all three on separate lines:
              u_i--;
              u_c--;
              u_s--;
              >>> LOCALS: read u_i, u_c, u_s after each decrement
    Step 5.   Declare  unsigned char   uc_max   =  255
    Step 6.   Increment:  uc_max++;
    Step 7.   Declare  unsigned char   uc_over  =  250
    Step 8.   Add 10 :   uc_over = uc_over + 10;

    Part B - Prefix vs postfix:

    Step 9.   Declare  int  pp_i  =  5
    Step 10.  Declare  int  pp_a  =  pp_i++
              >>> LOCALS after F10: read BOTH pp_a and pp_i
    Step 11.  Declare  int  pp_b  =  ++pp_i
              >>> LOCALS after F10: read BOTH pp_b and pp_i
  ============================================================ */

  /* --- write your Section 7 code here --- */

  return 0;
}
```

PART C — Canvas Quiz

22 questions · 1 pt each · no time limit · one attempt